MODULE 09 · DOMAIN 1 - DEVELOPMENT WITH AWS SERVICES

Amazon S3 for Developers

Presigned URLs, multipart uploads, conditional writes and events - the parts of S3 a developer actually writes code against.

DOMAIN 1 of 4 EXAM WEIGHT 32% MODULE 9 of 34

admin@sthithapragna.com

For a developer, S3 is not a filesystem. It is an **HTTP API with an object in it**, and almost everything follows from that.

Buckets, keys and prefixes

- A bucket name is **globally unique** across all AWS accounts, and lives in one Region.
- The **key** is the full path-like string. **invoices/2026/09/inv-1.pdf** is one key, not four levels.
- A **prefix** is just the leading portion of a key. It is what ListObjectsV2 filters on.
- There are no directories, so there is no atomic rename. Copy then delete.
- Object metadata is fixed at write time. Changing it means copying the object over itself.

> ON THE EXAM
>
> The flat namespace matters for performance too: request rates scale per prefix, so key design affects throughput exactly as it does in DynamoDB.

The size limits

LIMIT	VALUE
Object size	0 bytes to 5 TB on the exam. See the next slide
Single PUT upload	5 GB maximum
Recommended multipart threshold	About 100 MB
Multipart part size	5 MB minimum, 5 GB maximum. Last part may be smaller
Parts per upload	10,000
Console upload	160 GB

ON THE EXAM

5 GB is the number that decides questions. Anything larger than 5 GB cannot be uploaded in one PUT, so multipart is mandatory rather than optional.

Currency note: objects can now be 50 TB

- **The examinable answer:** the maximum S3 object size is **5 TB**.
- On **2 December 2025** AWS raised it to **50 TB**, a tenfold increase, in all Regions.
- It applies to every storage class and works with all S3 features.
- Above 5 TB, AWS directs you to the S3 Transfer Manager and the AWS Common Runtime.
- **Answer 5 TB on the exam.** Exam guide v2.1 predates this by a year.

> ON THE EXAM
>
> The 5 GB single-PUT limit did not change, and that is the one questions actually turn on. 5 TB against 50 TB is trivia by comparison - but you will meet 50 TB in the documentation.

Multipart upload

- Three steps: **CreateMultipartUpload**, then **UploadPart** repeatedly, then **CompleteMultipartUpload**.
- Parts upload **independently, in any order, in parallel**. A failed part retries alone.
- Each part returns an **ETag**, and completion submits the part number and ETag list.
- The object does not exist until completion. There is no partially visible object.
- The high-level SDK transfer managers do all of this for you automatically.

> ON THE EXAM
>
> The parallel retry is the real benefit. On a flaky connection, a failed 10 MB part costs one retry rather than restarting a 5 GB upload.

Incomplete multipart uploads

- An abandoned upload leaves its parts in the bucket, **billed as storage**, indefinitely.
- They are **invisible to ListObjectsV2**, because no object exists yet.
- Find them with **ListMultipartUploads**, remove them with **AbortMultipartUpload**.
- The real fix is a lifecycle rule: **AbortIncompleteMultipartUpload** after N days.
- AWS recommends this rule on every bucket that receives multipart uploads.

> ON THE EXAM
>
> A classic 'why is my bill higher than the sum of my objects' scenario. The storage is real, the objects are not listed, and only a lifecycle rule cleans it up automatically.

Presigned URLs

- A normal S3 URL plus a signature, granting one specific operation for a limited time.
- Works for **GET** and for **PUT**, so clients can upload directly to S3.
- The URL carries the permissions of **whoever signed it**, not of whoever uses it.
- The bucket stays entirely private. Nothing is made public.
- Anyone holding the URL can use it, as often as they like, until it expires.

> ON THE EXAM
>
> The standard upload pattern: the browser asks your API for a presigned PUT URL, then uploads straight to S3. Your compute never touches the bytes.

Why presigned URLs expire early

- A presigned URL dies when the **credentials that signed it** expire, whatever expiry you set.
- **IAM user credentials:** up to **7 days**, the SigV4 maximum.
- **An assumed role:** only as long as that session, often 1 hour.
- So a Lambda function signing a 7-day URL produces one that dies in about an hour.
- For genuinely long-lived access, use a **CloudFront signed URL** instead.

> ON THE EXAM
>
> A URL that expires far sooner than requested is almost always this. Lambda uses temporary role credentials, and no expiry setting can outlive them.

Presigned URL or CloudFront signed URL?

S3 PRESIGNED URL

Simple and direct

- Signed with IAM credentials
- Works for upload and download
- Maximum 7 days, often much less
- No CDN, no edge caching
- One line of SDK code

CLOUDFRONT SIGNED URL

Distribution and control

- Signed with a key pair you manage
- Download only
- Expiry can be far longer
- Cached at edge locations
- Can restrict by IP address

THE TIEBREAKER

Uploads always mean presigned URL - CloudFront cannot sign an upload. Long-lived or globally distributed downloads mean CloudFront.

Transfer Acceleration

- Uploads enter at the nearest **CloudFront edge location** and travel the AWS backbone.
- Enabled per bucket, then used via a distinct **s3-accelerate** endpoint.
- Helps most when clients are far from the bucket's Region.
- It costs extra, and adds nothing when the client is already near the Region.
- AWS provides a speed comparison tool to check it helps before you pay for it.

> ON THE EXAM
>
> 'Users worldwide uploading to a bucket in one Region, and it is slow' is the shape. Combine with multipart for large files.

Consistency and conditional writes

- S3 has had **strong read-after-write consistency** since December 2020, for all operations.
- Older material describes eventual consistency for overwrites and deletes. That is out of date.
- **If-None-Match: *** on PutObject writes only if the key does not exist. A conflict returns **412**.
- **If-Match** with an ETag writes only if the object is unchanged. Compare-and-swap.
- Extended to **CopyObject** in October 2025, with matching bucket policy condition keys.

> ON THE EXAM
>
> Conditional writes removed the old pattern of using DynamoDB as a lock table to coordinate S3 writers. Both patterns still appear as options.

Versioning

- Every write creates a new **version ID**. The old version remains and is still billed.
- A delete without a version ID writes a **delete marker** and hides the object.
- Removing the delete marker restores the object. That is the undelete.
- Deleting permanently means deleting a **specific version ID**.
- Versioning can be suspended but **never turned off**, and existing versions remain.

> ON THE EXAM
>
> Versioning plus a lifecycle rule on noncurrent versions is the standard pairing. Without the rule, storage grows forever.

Lifecycle rules

- **Transition** actions move objects between storage classes after N days.
- **Expiration** actions delete objects, or noncurrent versions, after N days.
- Rules can be scoped by **prefix**, by **object tag**, or by object size.
- **AbortIncompleteMultipartUpload** is a lifecycle action too. Set it everywhere.
- Lifecycle deletions are free and asynchronous. They are not immediate.

> ON THE EXAM
>
> Three separate cost leaks are all fixed here: old versions, forgotten objects, and abandoned multipart parts.

Object tags and cost attribution

- Up to **10 tags** per object, set at upload or added later.
- Lifecycle rules and replication rules can be **scoped by tag**, not just by prefix.
- IAM and bucket policies can condition on **s3:ExistingObjectTag** for fine-grained access.
- Tags appear in cost allocation reports, so one bucket can be billed by team.
- Tags are separate from user metadata, and are the only part you can change without copying.

> ON THE EXAM
>
> Tags are the way to apply different lifecycle or access rules inside one bucket when prefixes do not line up with the policy you want.

Encryption at rest

OPTION	WHO HOLDS AND MANAGES THE KEY
SSE-S3	S3 manages everything. The default on all new objects
SSE-KMS	A KMS key you choose. Auditable in CloudTrail, revocable
DSSE-KMS	Two independent layers of KMS encryption. For strict compliance
SSE-C	You supply the key on every request. AWS never stores it
Client-side	You encrypt before upload. S3 only ever sees ciphertext

ON THE EXAM

All new objects are encrypted with SSE-S3 by default. The question is never whether to encrypt, only which key and who controls it.

Access control

- **Block Public Access is on by default** and **ACLs are disabled by default** on new buckets.
- **Bucket policies** are the resource-based policy. They are how cross-account access is granted.
- IAM policies grant your principals access. Bucket policies grant others access.
- Block Public Access **overrides** any policy that would make the bucket public.
- For CloudFront in front of S3, use **origin access control** rather than a public bucket.

> ON THE EXAM
>
> Any option that solves a problem by making a bucket public is almost certainly wrong. Presigned URLs or OAC are the intended answers.

CORS

- A JSON configuration on the bucket listing allowed origins, methods and headers.
- Without it, a browser on another domain is blocked **by the browser**, not by S3.
- The browser sends an **OPTIONS** preflight first for anything beyond a simple request.
- **ExposeHeaders** is needed for JavaScript to read response headers like ETag.
- Presigned PUT uploads from a browser require CORS. This is the usual cause of failure.

> ON THE EXAM
>
> A presigned URL that works in curl but fails in a browser is a CORS problem every time. The signature is fine; the browser is enforcing policy.

Events out of S3

- **Event notifications** send directly to Lambda, SQS or SNS. Asynchronous invocation.
- One destination per event type and prefix. Fan-out needs SNS in the middle.
- **EventBridge**, enabled per bucket, publishes every event to the default bus.
- EventBridge gives many targets, richer filtering, plus archive and replay.
- Notification delivery is typically seconds but is **not guaranteed to be ordered**.

> ON THE EXAM
>
> One consumer, keep it simple: notifications. Several consumers, or you need filtering and replay: EventBridge. Module 5 covered the Lambda side.

Reading less than the whole object

- **Byte-range fetches** use the HTTP **Range** header to read part of an object.
- Useful for headers of large files, and for parallel downloads of separate ranges.
- **S3 Object Lambda** runs a function that transforms an object as it is retrieved.
- One stored object can then be redacted, converted or filtered per caller.
- Object Lambda adds Lambda cost and latency to every GET that goes through it.

> ON THE EXAM
>
> Object Lambda is the answer when different consumers need different views of the same object and you do not want to store several copies.

Currency note: S3 Select is closed

- **The examinable answer:** S3 Select runs SQL against a single object without downloading it.
- **S3 Select and S3 Glacier Select closed to new customers on 25 July 2024.**
- Existing customers continue to use it, with security and availability updates.
- AWS now directs you to **Amazon Athena**, **S3 Object Lambda**, or client-side filtering.
- It remains a correct exam answer even though you cannot sign up for it.

> ON THE EXAM
>
> The clearest example of the revision gap in this course: a service you cannot purchase, that is still the right answer on the exam.

S3 performance

- At least **3,500 PUT, COPY, POST or DELETE** requests per second per partitioned prefix.
- At least **5,500 GET or HEAD** requests per second per partitioned prefix.
- The limits are **per prefix**, so spreading keys across prefixes multiplies throughput.
- S3 also repartitions automatically as request patterns grow.
- Sequential key prefixes such as timestamps concentrate load, exactly as in DynamoDB.

> ON THE EXAM
>
> The same lesson as module 7. Load spreads across the leading portion of the key, so keys beginning with a date concentrate traffic.

Two of these are diagnosis: something works in one context and fails in another. Ask **what is different** about the failing case.

A web application lets users upload video files of up to 4 GB. Currently the browser posts the file to an EC2-backed API, which buffers it and forwards it to Amazon S3. The API servers are running out of memory and disk, and upload latency is poor for distant users.

The S3 bucket must remain private. The team wants uploads to stop passing through their compute entirely.

How should the upload be redesigned?

A. Make the S3 bucket public for writes and have the browser PUT directly to it

B. Increase the EC2 instance size and add an Application Load Balancer

C. Store the uploads in Amazon EFS mounted on the EC2 instances, then sync to S3

D. Have the API return a presigned PUT URL, and let the browser upload directly to S3 using multipart

◆ PAUSE AND DECIDE

How should the upload be redesigned?

A Public bucket

Directly contradicts the requirement that the bucket stay private, and a publicly writable bucket is a serious security failure. Almost never a correct answer.

B Bigger instances plus an ALB

Scales the problem rather than removing it. The bytes still flow through your compute, and you pay for that traffic twice.

C EFS then sync

Adds a second storage system and still routes every byte through EC2. More moving parts, same bottleneck.

D Presigned PUT plus multipart

The bucket stays private, the bytes never touch your compute, and multipart handles 4 GB with resumable parts.

THE SIGNAL

'Must remain private' and 'stop passing through our compute' look contradictory until you remember a presigned URL grants access without making anything public.

A Lambda function generates presigned GET URLs for report files and emails them to customers, with an expiry set to 7 days. Customers report that the links stop working after about an hour, long before the stated expiry.

The same code, run locally by a developer using their own IAM user access keys, produces URLs that work for the full 7 days.

What explains the difference, and what should the developer do?

A The Lambda execution role provides temporary credentials that expire with the session, so the URL dies with them. Use CloudFront signed URLs for long-lived links

B The Lambda function's timeout is too short, so the URL is not fully signed

C S3 caps all presigned URLs at one hour regardless of the requested expiry

D The bucket lifecycle policy is expiring the objects after one hour

◆ PAUSE AND DECIDE

What explains the difference, and what should the developer do?

A Role credentials expire; use CloudFront

A presigned URL cannot outlive the credentials that signed it. Lambda uses an assumed role, so the URL dies with the session. CloudFront signed URLs use a managed key pair and have no such ceiling.

B Function timeout

Signing is a local cryptographic operation taking milliseconds. A timeout would fail the invocation outright, not produce a short-lived URL.

C S3 caps at one hour

No such cap exists. SigV4 allows up to 7 days, which the developer's local run demonstrates.

D Lifecycle expiry

That would delete the objects, so the developer's own URLs would fail too. They do not.

THE SIGNAL

The local run working is the whole clue. Same code, same bucket, same expiry - only the credentials differ, so the credentials are the cause.

> A distributed job writes result files to S3. Several workers can occasionally be assigned the same job, and both then write to the same key. The team needs the first write to win, and later writers to detect that the result already exists rather than overwrite it.
>
> They do not want to introduce another service, and the workers have no shared coordination mechanism.

How should this be implemented?

A Enable versioning on the bucket so every write is preserved

B Use PutObject with the If-None-Match header set to *, and handle the 412 response

C Have each worker call HeadObject first and skip the write if the object exists

D Write to a DynamoDB lock table before writing to S3

◆ PAUSE AND DECIDE

How should this be implemented?

A Versioning

Preserves both writes rather than preventing the second. The last writer is still the current version, so readers get the wrong result.

B If-None-Match on PutObject

S3 evaluates the condition atomically. The first write succeeds, the second gets 412 Precondition Failed and knows it lost. No extra service.

C HeadObject then write

A check-then-act race. Both workers can see no object and both proceed. This is the bug conditional writes were built to remove.

D DynamoDB lock table

It works, and it was the standard pattern before August 2024. It adds the service the scenario explicitly rules out.

THE SIGNAL

Option C is the tempting one, and it is a textbook race condition. Only a condition evaluated inside S3 is atomic.

> A finance team reports that the S3 bill for a media bucket is roughly 40% higher than the total size of the objects it contains. The bucket receives large uploads from a mobile app, which frequently loses connectivity mid-upload. Versioning is not enabled.
>
> Listing the bucket accounts for only the expected object storage. The team wants the excess removed and prevented from recurring.

What is the cause, and what should the team configure?

A. Old object versions are accumulating. Add a lifecycle rule to expire noncurrent versions

B. The objects are in the wrong storage class. Add a lifecycle transition to S3 Standard-IA

C. Abandoned multipart upload parts are being stored. Add a lifecycle rule with AbortIncompleteMultipartUpload

D. Request charges account for the difference. Enable S3 Transfer Acceleration to reduce them

◆ PAUSE AND DECIDE

What is the cause, and what should the team configure?

A. Expire noncurrent versions

The scenario states versioning is not enabled, so there are no noncurrent versions to expire. Eliminated by one sentence.

B. Transition to Standard-IA

Would reduce cost per gigabyte, and leaves the mystery storage in place. It treats the symptom while the leak keeps growing.

C. AbortIncompleteMultipartUpload

Interrupted uploads leave parts that are billed but never listed, which explains storage that listing cannot account for. The lifecycle rule removes them and prevents recurrence.

D. Transfer Acceleration

Speeds uploads and costs more, not less. Request charges are also unrelated to a storage discrepancy.

THE SIGNAL

Three clues converge: storage that listing cannot see, large uploads, and frequent disconnections. Only incomplete multipart parts fit all three.

What to carry forward

- **5 GB** is the single PUT limit. Above that, multipart is mandatory, not optional.
- A presigned URL **cannot outlive the credentials that signed it**.
- Conditional writes give atomic create-if-absent and compare-and-swap, since 2024.
- Lifecycle rules fix three cost leaks: old versions, stale objects, abandoned parts.
- Making a bucket public is almost never the answer. Presigned URLs or OAC are.

> ON THE EXAM
>
> Next: caching, relational data, and choosing between DynamoDB, RDS, Aurora and ElastiCache when several would work.